BACSE202_DATABASE-SYSTEMS

PROJECT REPORT

on

Receipt Aware Dual State Database Auditor

Prepared by

Swarnava Chaudhuri - 25BCE2463

Shivani Patodia - 25BCE2403

Shreya Verma - 25BCE2425



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

Vellore Institute of Technology, Vellore.

October 20, 2026

Table of Contents

Abstract 3

1. Introduction 4

2. Literature Study 5

3. Objectives 6

4. Design 7

5. Implementation 8

5.1 Experimental findings 9

6. Conclusion 10

References 11

Abstract

A database audit log is useful only while its evidence can be trusted. If a privileged user can change both a stored row and its local history, a matching log alone does not establish integrity. This project extends a dual-state database auditor with a receipt reconciliation laboratory. Each accepted write records its before and after state, revision and previous receipt hash in a witness store before the business transaction is committed. Verification checks the witness history against the database audit entry and live row, then checks the latest receipt against a Merkle checkpoint where one is available. A legitimate write awaiting its checkpoint is reported as pending rather than tampered. Inconsistent evidence prevents further writes to the affected record and blocks checkpoint creation. The implementation includes a local browser dashboard, controlled fault injection, automated tests and reproducible experiments. All 32 tests passed in the recorded run. The experiments used synthetic financial rows in SQLite, with a local file representing the witness trust domain. They do not establish production isolation or PostgreSQL performance. The work offers a testable prototype and a specific candidate contribution for prior-art assessment; it does not establish patentability.

1. Introduction

Consider a fee record stored as 250.00. A database administrator changes it to 999999.00 and edits the audit entry to match. A comparison between those two database objects can now succeed even though the record was changed outside the application. An independent witness retains the earlier state and gives the verifier a second reference. The important question is whether the current row agrees with evidence that the database administrator cannot rewrite.

The starting repository contains a PostgreSQL audit pipeline, SHA-256 event hashes, Merkle proofs, a local Hardhat anchor and a Streamlit interface [6]. These are established techniques. The extension in this report adds a separate, dependency-free laboratory for exploring receipt reconciliation and demonstrating its failure cases. It reuses the repository hash and Merkle modules rather than presenting them as a new invention.

Scope and assumptions

The attacker in the experiments can alter business records and database audit entries. The witness database is assumed to remain available and unchanged. The prototype places both databases on one computer, so this separation is simulated. An operating-system administrator who controls both files is outside the demonstrated protection. The application endpoint records accepted writes; it does not decide whether a caller has a valid business reason to change a record. Authentication, role-based approval and a remotely administered witness are future deployment requirements.

The system detects disagreement. It does not prevent a direct SQL edit, recover the original row automatically or prove that an unwitnessed state was never changed. A write is only checkpointed when the evidence agrees. Missing evidence is reported explicitly rather than labelled valid.

2. Literature Study

Veritas describes shared verifiable database tables for parties that need evidence about their shared data [1]. Verity studies blockchain-based detection of insider changes to SQL data [2]. These works show that database integrity can be checked through evidence beyond ordinary access controls. Their existence means that a broad claim of combining a database with cryptographic verification would be weak.

Microsoft patent publication US20210377037A1 describes asynchronous relational-data integrity using transaction history, cryptographic structures and receipts [3]. It is particularly close to the original project architecture. US10261711B1 also discusses stored-data non-tampering proofs and pending Merkle updates [4]. Therefore, merely adding an asynchronous worker, a receipt or an external root is not enough to establish a new invention. SQL Server ledger documentation provides a further practical reference for tamper-evident data and external digests [5].

Candidate contribution

The extension makes the verification decision explicit: a witnessed change awaiting a checkpoint is pending; an altered row is inconsistent; a rewritten audit event is inconsistent; and a witnessed deletion can be verified after anchoring. The checkpoint operation first reconciles the live rows and audit evidence with the witness receipts. It refuses to seal a known disagreement. This workflow is the candidate contribution to investigate further. No claim is made that these rules are absent from every earlier patent or publication.

Prior art assessment

The next search should compare claims covering witness-first receipts, revision-linked transitions, reconciliation before checkpointing, and pending-state classification. The library search must include patent families and relevant Indian publications. The references here are genuine starting points, not an exhaustive novelty or freedom-to-operate opinion. A patent proposal should proceed only if a narrower technical distinction survives that assessment.

3. Objectives

The implementation has five objectives. First, record accepted application changes in a witness store separately from the business database. Second, distinguish a consistent pending change from a mismatch. Third, validate a Merkle proof before reporting a checkpointed state as verified. Fourth, refuse checkpoint creation when row or audit evidence disagrees with witness history. Fifth, make each outcome reproducible through controlled faults and recorded tests.

The demonstration is successful when an ordinary update remains pending until anchoring, an unlogged edit is detected, a coordinated row-and-audit rewrite is detected, and a legitimate deletion is distinguishable from an unlogged deletion. Persistence is checked by reopening the databases. Input checks reject non-finite amounts and precision beyond two decimal places.

Production throughput, resistance to full host compromise, zero-knowledge verification, public-chain finality and automatic recovery are outside the evaluated scope. The project does not claim the speed of an unaudited database. Recording a witness receipt introduces synchronous work, while checkpoint creation is a separate operation.

4. Design

The laboratory has three logical components. The business store contains records and audit entries. The witness store contains ordered receipts and checkpoint documents. The verifier reads both and returns a structured verdict. A loopback HTTP server exposes these actions through a dashboard. Both stores use SQLite in this tested mode. The pre-existing PostgreSQL and Hardhat components are a separate mode.

For each write, the receipt contains record ID, revision, before-state, after-state and the hash of the previous receipt for that record. The witness assigns a sequence number. The business transaction then updates the row and inserts the matching audit event. This is a witness-first protocol, not a distributed atomic commit. If the process fails between the two commits, the retained receipt can expose missing evidence on the next verification.

Verification first checks the receipt chain and database audit entries. It compares the current row with the latest witnessed after-state. If these checks succeed, it looks for the latest receipt in a checkpoint. The leaf is SHA-256 over canonical JSON containing the receipt sequence and event. Ordered sibling hashes reconstruct the checkpoint root. A missing checkpoint produces PENDING_ANCHOR; a matching checkpoint produces VERIFIED, or DELETED_VERIFIED for an accepted deletion.

Checkpoint creation reconciles every tracked record and compares the database audit sequence set with the witness sequence set. Only an agreeing set can be anchored. A single inconsistent record blocks the batch. This conservative policy is simple to explain, but can reduce availability. Isolating independent records into separate checkpoint groups is a possible extension.

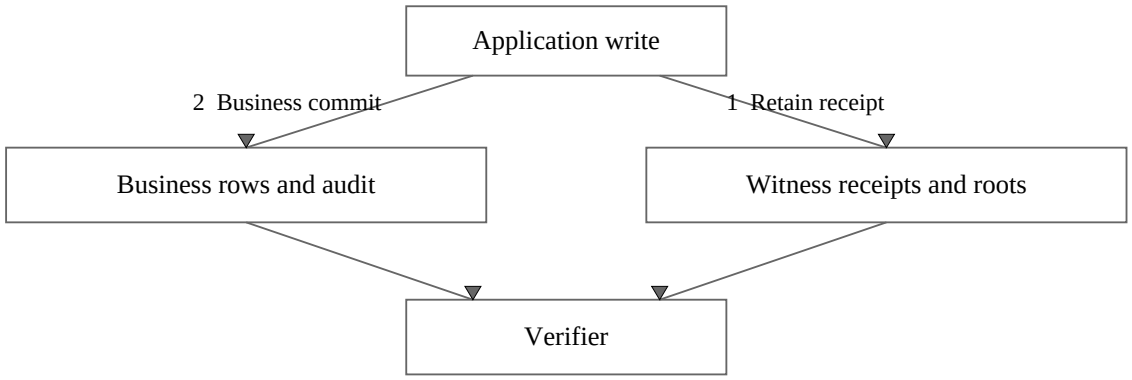


Figure 1. The verifier reconciles evidence from both logical stores.

5. Implementation

The added implementation is in demo/engine.py, demo/server.py, demo/index.html and demo/experiments.py. It uses Python standard-library SQLite and HTTP modules, plus the existing hash and Merkle code. No Docker, blockchain wallet or paid service is needed for this mode. Start it with python -m demo.server --port 18877 and open http://127.0.0.1:18877. The server binds to loopback and requires a per-process token for mutation requests. It is a local teaching tool rather than a public web service.

Automated validation ran on Windows with Python 3.13.15. The suite contains 16 original hashing/Merkle cases and 16 new reconciliation cases. All 32 passed. New cases include legitimate pending updates, direct edits, coordinated row-and-audit rewrites, unlogged deletions, accepted deletions, missing audit entries, checkpoint blocking, damaged proof paths, rollback, restart persistence, orphaned receipts, invalid amounts, empty state, odd-sized batches and unwitnessed inserts. The raw test log is included in evidence/unit-tests.txt.

Source code and reproducibility

Run python -m unittest discover -s tests -v to reproduce the suite. Run python -m demo.experiments to generate a new measurement file. Each fault experiment uses a temporary database. The delivered source archive excludes private configuration, installed dependencies and live demo data.

5.1 Experimental findings

Controlled faults produced the following recorded outcomes: unlogged edit, TAMPER_DETECTED; row-and-audit rewrite, AUDIT_TAMPERED; unlogged deletion, TAMPER_DETECTED; audit erasure, EVIDENCE_MISSING. These outcomes establish behaviour for the tested faults under the stated trust assumption. They are not a detection-rate estimate for arbitrary attacks.

Records	Write median	Checkpoint	Verify median	Siblings
10	5.519 ms	8.474 ms	0.509 ms	4
100	6.085 ms	110.088 ms	1.061 ms	7
300	6.842 ms	4279.445 ms	12.059 ms	9

Measurements used Python 3.13.15 on Windows. Each size has one run, with verification over up to 30 records. The data consists of synthetic amounts. No production dataset or real customer information was used.

The longer checkpoint time at 300 records exposes an implementation limit: verification repeatedly scans receipt history and checkpoint documents. A short Merkle path does not make the whole verifier logarithmic. These timings include local SQLite commits, use one run per size and have no network delay or PostgreSQL baseline. They support a small classroom demonstration, not a scalability or low-overhead claim.

6. Conclusion

The prototype demonstrates that a changed row and a rewritten database log can still disagree with a separately retained receipt. It also distinguishes legitimate changes waiting for a checkpoint from demonstrated inconsistencies. The tests passed for the listed scenarios, and the dashboard exposes real receipts, hashes, proofs and verdicts rather than prewritten results.

The work is currently a local laboratory prototype. The next engineering step is to move the witness behind a separately administered service with authenticated requests, durable keys and an explicit recovery protocol. Other priorities are indexed witness lookup, checkpoint recovery after process failure, denial-of-service handling and integration of the reconciliation rules into the PostgreSQL pipeline. Additional tests should cover concurrent clients, interrupted writes and larger histories.

The originality question remains open. Existing patents already cover much of the broad architecture. A narrower claim around the implemented reconciliation rules would need a claim-by-claim search and an assessment of non-obviousness. Until that process and the required course approval are complete, this report presents the work as an implementation-based project with a candidate patent direction.

References

[1] A. Allen et al., "Veritas: Shared Verifiable Databases and Tables in the Cloud," CIDR, 2019. https://www.vldb.org/cidrdb/papers/2019/p111-gehrke-cidr19.pdf

[2] S. S. Srivastava et al., "Verity: Blockchains to Detect Insider Attacks in DBMS," arXiv:1901.00228, 2019. https://arxiv.org/abs/1901.00228

[3] P. Antonopoulos, J. J. Szymaszek, R. Kaushik and C. J. Cunningham, "Asynchronously determining relational data integrity using cryptographic data structures," US20210377037A1, published Dec. 2, 2021. https://patents.google.com/patent/US20210377037A1/en

[4] "Proof of non-tampering for stored data," US10261711B1, 2019. https://patents.google.com/patent/US10261711B1/en

[5] Microsoft, "Ledger overview," SQL Server documentation. https://learn.microsoft.com/en-us/sql/relational-databases/security/ledger/ledger-overview Accessed Oct. 20, 2026.

[6] shverma524, "dual-state-database-auditor," project source repository. https://github.com/shverma524/dual-state-database-auditor Local source inspected Oct. 20, 2026.

[7] Project source and execution evidence, demo/engine.py, tests/test_receipts.py, evidence/unit-tests.txt and evidence/experiments.json, recorded Oct. 20, 2026 IST.